

AWS Well-Architected Framework Review for ROCA Middle-Tier Flows: Orchestration, Scheduling, and Notification Architecture (Batch 3 of 4)

1 Executive Summary

ROCA's four middle-tier flows — Volunteer Assignment, Event Cancellation, Recurring Event Management, and Push Notification Engagement — exhibit critical reliability and security gaps that compound six unremediated Batch 1 deficiencies, most notably embedded state machines in application code, absent dead-letter queues, and missing WAF/GuardDuty deployment. The Event Cancellation flow poses the highest immediate risk: its multi-step cascade (notification, refund, reassignment) lacks saga orchestration via Step Functions, meaning a failure mid-cascade leaves the system in an unrecoverable partial state with no compensating transactions to restore consistency [1, 2]. Silent message loss from absent DLQs across all SQS queues [3] amplifies this risk, as failed refund or notification messages disappear without trace.

Two additional flow-specific risks demand urgent attention. The Push Notification flow's four-channel delivery architecture has no systematic SES bounce handling or complaint routing, threatening sender reputation degradation if bounce rates exceed 5% [4, 5], and any remaining legacy FCM endpoints must complete HTTP v1 API migration immediately to avoid delivery disruption [6]. The Recurring Event flow's RFC 5545 RRULE parser permits unbounded expansion without enforced COUNT or UNTIL constraints, risking Lambda timeout exhaustion under production load [7, 8].

The remediation roadmap totals 80–95 engineering days across three tiers. The P0-Critical tier (~27 days) should deploy DLQs, implement the cancellation saga pattern in Step Functions, configure SES bounce handling, and close inherited security gaps (WAF, GuardDuty, JWT rotation). Completing these items within 30 days materially reduces the compounding risk that currently propagates across all four flows before P1 architectural migration and P2 cost optimization proceed.

2 Introduction and Scope

This report presents the third batch of ROCA's AWS Well-Architected Framework Review, evaluating four middle-tier flows that form the operational backbone of the platform: Volunteer Assignment and Confirmation, Event Cancellation and Notification, Recurring Event Management, and Push Notification Engagement. These flows are characterized by complex state machines, multi-service orchestration, timed cascades, and real-time delivery requirements that introduce architectural risks distinct from the foundational flows reviewed in Batches 1 and 2.

The review evaluates each flow against all six Well-Architected pillars — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability — while cross-referencing the fifteen unremediated gaps identified in Batch 1 and the escalations added in Batch 2. A recurring theme emerges: the compounding effect of embedded state machines in application code, absent dead-letter queues, and missing Step Functions orchestration now propagates across all four Batch 3 flows, amplifying risk at each layer. New risks specific to this batch include saga pattern requirements for cancellation cascades with compensating transactions [1, 2], compute cost implications of the auto-schedule matching algorithm [9], RRULE materialization performance under unbounded expansion [7], multi-channel notification delivery reliability with bounce handling [10, 4], SQS message ordering for cascading operations [11], and WebSocket connection scaling at concurrency [12].

The report concludes with a prioritized remediation roadmap spanning approximately 80–95 engineering days across three priority tiers, sequenced to address critical reliability and security gaps first before optimizing performance and cost.

3 Unremediated Batch 1/2 Gaps and Compounding Risk

Batch 1 identified fifteen architectural gaps, including embedded state machines in application code, missing Step Functions orchestration, absent JWT rotation, no dead-letter queues on SQS, no WAF or GuardDuty deployment, no EventBridge for event-driven decoupling, unencrypted PII at rest, and no X-Ray distributed tracing. Batch 2 confirmed that all six critical Batch 1 gaps remain unremediated and escalated WAF, GuardDuty, and JWT rotation to P0-Critical priority due to PCI DSS v4.0.1 compliance requirements. Batch 2 additionally surfaced RSVP race conditions, RRULE DST parsing risks, missing Stripe key rotation, and the absence of a cancellation saga pattern.

The compounding effect across Batch 3 is significant. All four middle-tier flows inherit the embedded state machine anti-pattern: the Volunteer Assignment flow manages an eight-state assignment lifecycle in application code, the Event Cancellation flow orchestrates a multi-step cascade without saga compensation, the Recurring Event flow tracks series state transitions without formal orchestration, and the Push Notification flow manages a six-state delivery lifecycle without externalized state tracking. The absence of DLQs means that failed messages in any of

these flows — cancellation refund failures, notification delivery bounces, swap request timeouts — are silently lost rather than isolated for investigation and replay [3]. The missing WAF and GuardDuty leave all four flows exposed to API abuse and threat detection blind spots, a gap that is particularly acute for the Push Notification flow’s WebSocket endpoints and the Event Cancellation flow’s refund processing.

4 Flow 1: Volunteer Assignment and Confirmation

4.1 Auto-Schedule Algorithm Compute Risk

The six-step volunteer assignment flow centers on an auto-schedule algorithm that scores candidates across four weighted dimensions: skill match (40%), availability (30%), workload balance (20%), and volunteer preference (10%). This compute-intensive matching currently runs in Lambda without memory right-sizing, which directly impacts CPU allocation. Lambda memory is configurable from 128 MB to 10,240 MB, and at 1,769 MB a function receives the equivalent of one full vCPU [9]. Without profiling to find the optimal memory setting where increased CPU offsets shorter duration costs, the algorithm likely runs either over-provisioned (wasting cost) or under-provisioned (extending duration and increasing latency). AWS Compute Optimizer provides machine-learning-based memory size recommendations by continuously monitoring historical performance metrics [13], and should be enabled immediately for all scheduling Lambda functions.

Further cost reduction is available through Graviton2 (arm64) architecture, which can achieve significantly better price and performance for compute-intensive applications — Lambda functions using arm64 benefit from a larger L2 cache per vCPU that improves latency for data processing workloads [14]. Combining right-sizing with Graviton2 and caching frequently accessed data (skill profiles, availability windows) in Lambda /tmp storage or ElastiCache would reduce redundant computation across successive scheduling runs [15].

4.2 State Machine and Swap Workflow Gaps

The assignment state machine (INVITED → ACCEPTED → CONFIRMED → COMPLETED, with DECLINED, SWAP_REQUESTED, SWAP_APPROVED, and REASSIGNED branches) remains embedded in application code — the same anti-pattern flagged in Batch 1. This should be migrated to Step Functions. Given that auto-schedule runs and swap workflows typically complete within five minutes, Express Workflows are the appropriate choice: they provide \$1.00 per million invocations versus \$25 per million for Standard Workflows, with throughput of up to 100,000 state transitions per second [16]. The key tradeoff is that Express Workflows use at-least-once semantics rather than exactly-once, so all scheduling and swap operations must be designed to be idempotent [17]. The swap request workflow currently lacks compensating transactions — if a swap is approved but the replacement volunteer’s confirmation fails, there is no automated rollback to restore the original assignment. Step Functions’ built-in Retry with exponential backoff and Catch with ResultPath can route errors to compensation branches, reducing custom error-handling boilerplate [18].

5 Flow 2: Event Cancellation and Notification Cascade

5.1 Saga Pattern for Cancellation Orchestration

The four-step event cancellation flow — impact analysis (affected attendees, volunteers, payments), notification cascade, refund processing, and volunteer reassignment — is the most architecturally complex flow in Batch 3 and the most exposed to data inconsistency. A cancellation that successfully processes refunds but fails during volunteer reassignment leaves the system in a partially completed state with no automated recovery path. This flow requires the saga orchestration pattern, where Step Functions serves as the central coordinator preserving data integrity across distributed transactions that span multiple services [1]. Each cancellation step should be defined as a Task state with a corresponding compensating Task state wired via Catch blocks that execute in reverse order on failure — if any step fails, Step Functions orchestrates compensating transactions (reverse refund, cancel notification, undo reassignment) to restore consistency [2].

All compensating transactions must be idempotent, since retries are inherent in distributed saga flows and each compensation Lambda must safely handle duplicate invocations without side effects [2]. Step Functions’ built-in retry and error-handling capabilities should be leveraged to minimize custom try/catch and backoff logic within the Lambda functions themselves [18].

5.2 Message Ordering and Timed Cascade

The notification cascade follows a timed sequence: T+0 in-app/push notification, T+5 minutes email, T+15 minutes SMS, and T+1 hour refund status update. This timed progression maps naturally to Step Functions Wait states,

which can pause execution for precise durations before triggering the next notification channel. For the underlying message transport, SQS FIFO queues are essential where operation order matters: FIFO guarantees that messages within the same message group are processed in strict order and provides exactly-once processing via deduplication [11]. Standard queues offer higher throughput but only best-effort ordering, making them suitable for independent fan-out notifications where order is non-critical but inappropriate for the sequential cancellation cascade.

Dead-letter queues must be attached to both SQS source queues and Lambda functions with a `maxReceiveCount` redrive policy, so messages that repeatedly fail compensation are isolated for investigation and later redrive rather than blocking the pipeline [3]. Step Functions integration can additionally route failed state executions and their context into a DLQ for replay [18].

6 Flow 3: Recurring Event Management

6.1 RRULE Parsing and Unbounded Expansion Risk

The four-step recurring event flow parses RFC 5545 RRULE definitions to generate event instances, supports edit scopes (`this_only`, `this_and_future`, `all`), manages exceptions (`EXDATE`/`RDATE`), and tracks series state (`ACTIVE` → `PAUSED` → `ENDED`/`CANCELLED`). The most significant performance risk is unbounded RRULE expansion: RFC 5545 permits RRULE without `COUNT` or `UNTIL`, meaning a rule like `FREQ=DAILY` generates infinite occurrences [7]. The specification states that `UNTIL` and `COUNT` must not appear together, so a rule ends one of exactly three ways: never, after `N` occurrences, or at a date [19]. Within Lambda's 15-minute maximum timeout and up to 10,240 MB memory [8], an unbounded RRULE with multiple `BY*` modifiers can exhaust execution time and memory. Input validation must enforce that all user-created RRULEs include either `COUNT` or `UNTIL`, and all query-time expansion must be bounded to a fixed window.

6.2 DST Edge Cases

RFC 5545 recommends that `DTSTART` be specified as local time with a `TZID` reference so that instances maintain the same wall-clock time across DST transitions [20]. Libraries like `rrule.js` and `python-dateutil` have known bugs where occurrences shift by one hour after DST boundaries when `pytz` or UTC-only timestamps are used [21, 22]. ROCA must ensure all recurring event `DTSTART` values use `TZID`-anchored local time rather than bare UTC to prevent systematic time drift across series instances.

6.3 Materialization Strategy

The recommended approach is hybrid materialization: lazy expansion within bounded query windows (approximately 90 days) for reads, with eager persistence only for modified or exception instances. This avoids the write amplification and storage costs of full eager materialization while keeping read latency manageable. Google Calendar uses this server-side lazy expansion approach, and CalDAV delegates expansion to the client [19]. For the data layer, DynamoDB single-table design with `PK=SERIES#id` and SK patterns for master, exceptions, and materialized instances — combined with a GSI on `NextOccurrenceDate` — enables efficient date-range queries [23]. DynamoDB Streams can trigger Lambda to eagerly materialize the next window of instances on series update [24]. Alternatively, PostgreSQL's `generate_series()` function can expand recurrences at query time without storing individual rows [25], and materialized views with concurrent refresh support read-heavy dashboard queries [26].

7 Flow 4: Push Notification Engagement

7.1 Multi-Channel Delivery Architecture

The push notification flow delivers across four channels — FCM (Android), APNs (iOS), AWS SES (email), and AWS SNS SMS — with a smart delivery engine that enforces quiet hours, frequency caps (5 push/day, 2 email/day), channel optimization via engagement history, and ML-predicted optimal timing. The notification state machine tracks each message through `QUEUED` → `SCHEDULED` → `SENT` → `DELIVERED` → `READ` → `ACTED`, with `BOUNCED` and `DISMISSED` as terminal branch states triggered by provider feedback [27]. The architecture should decouple notification generation from delivery using SQS, with workers processing jobs asynchronously through each state [27]. The push notification service acts as an orchestration layer — iOS notifications route through APNs, Android through FCM — and never delivers directly [28].

A critical operational requirement is the FCM HTTP v1 API migration. AWS SNS now requires FCM HTTP v1 API for all device types including Apple and Webpush destinations, with the migration deadline having passed in June 2024 [6]. Any remaining legacy FCM endpoints must be migrated immediately to avoid delivery disruption.

7.2 Bounce Handling and Sender Reputation

For SES email delivery, hard bounces (permanent) should immediately transition to BOUNCED and suppress the address — SES does not retry hard bounces, and repeated attempts to hard-bounced addresses must be avoided [10]. Soft bounces (transient) can be identified via the `bounceSubType` field in SNS notifications and retried with exponential backoff [29]. Separate SNS topics should be configured for bounces and complaints to isolate notification channels, with SQS subscriptions ensuring notifications persist until processed [4]. Maintaining a bounce rate under 5% and a complaint rate under 0.1% is essential to preserve sender reputation [5].

For push channels, delivery status logging should be enabled on SNS platform applications — logs are sent to CloudWatch and contain provider response codes from push platforms [30]. Device tokens must be stored with endpoint ARNs and timestamps, with stale tokens removed and corresponding endpoints deleted; SNS automatically disables endpoints when a notification service reports the token as invalid [31, 32].

7.3 Frequency Capping and ML-Predicted Timing

The frequency capping layer must enforce hard constraints including daily per-user caps, quiet-hours windows, opt-out preferences, and regulatory rules, with support for a batched digest path that aggregates low-priority candidates into a single notification at the user's optimal hour [33]. The ML timing engine should score every non-transactional notification for a send/don't-send decision maximizing long-horizon engagement, route approved sends to the best channel based on content type and user channel history, and pick send-time windows inside the user's active-hours profile [33]. For cold-start users with no engagement history, demographic cohort priors provide a conservative fallback [33].

7.4 WebSocket Connection Scaling

The in-app notification channel uses API Gateway WebSocket APIs, which do not enforce a hard concurrent connection quota — the maximum is determined by the rate of new connections per second (default 500, increasable via Service Quotas) and the 2-hour maximum connection duration, theoretically supporting up to 3,600,000 concurrent connections at sustained maximum connect rate [12]. However, the idle connection timeout is 10 minutes and is not configurable, and maximum connection duration is 2 hours and is also not configurable [12]. Clients must implement heartbeat/ping messages at intervals under 10 minutes to prevent disconnection, and must handle automatic reconnection with exponential backoff after the 2-hour hard cutoff. Messages exceeding 128 KB close the connection with code 1009 [34].

Backend services send notifications to connected clients via POST requests to the `@connections/{connection_id}` callback URL, which requires IAM SigV4 signing [35]. Connection IDs should be stored in DynamoDB, with fan-out iterating over active connections and handling stale `GoneException` errors to clean up disconnected clients. The integration timeout for Lambda is 50ms–29 seconds [12], so high-scale fan-out should use Lambda concurrency controls and consider batching `@connections` calls to avoid throttling. WebSocket API pricing is \$1.00 per million messages and \$0.25 per million connection minutes [36], with a free tier of 1 million messages and 750,000 connection minutes per month for 12 months [37].

8 Six-Pillar Well-Architected Assessment

The following heatmap summarizes risk severity across all four flows and six Well-Architected pillars, synthesizing the flow-specific findings above.

Chart Explanation: This heatmap visualizes the assessed risk severity for each of the four middle-tier flows across the six Well-Architected pillars. Critical ratings (red) indicate immediate architectural gaps requiring P0 remediation — notably the Event Cancellation flow's Operational Excellence and Reliability gaps due to missing saga orchestration [1, 2] and absent DLQs [3], the Recurring Event Management flow's Performance Efficiency risk from unbounded RRULE materialization [7], and the Push Notification flow's Reliability exposure from missing bounce handling [10, 4] and FCM v1 migration [6].

Operational Excellence is degraded across all flows by embedded state machines in application code, the absence of X-Ray distributed tracing, and no EventBridge for event-driven decoupling. The Event Cancellation flow is rated Critical because the multi-step cascade has no formal orchestration, making failure diagnosis and recovery manual and error-prone.

Security inherits the Batch 1/2 P0-Critical gaps: JWT rotation remains absent, WAF and GuardDuty are not deployed, and PII encryption gaps persist. The Push Notification flow adds channel-specific security concerns around FCM v1 migration [6] and IAM SigV4 signing for WebSocket callback URLs [35].

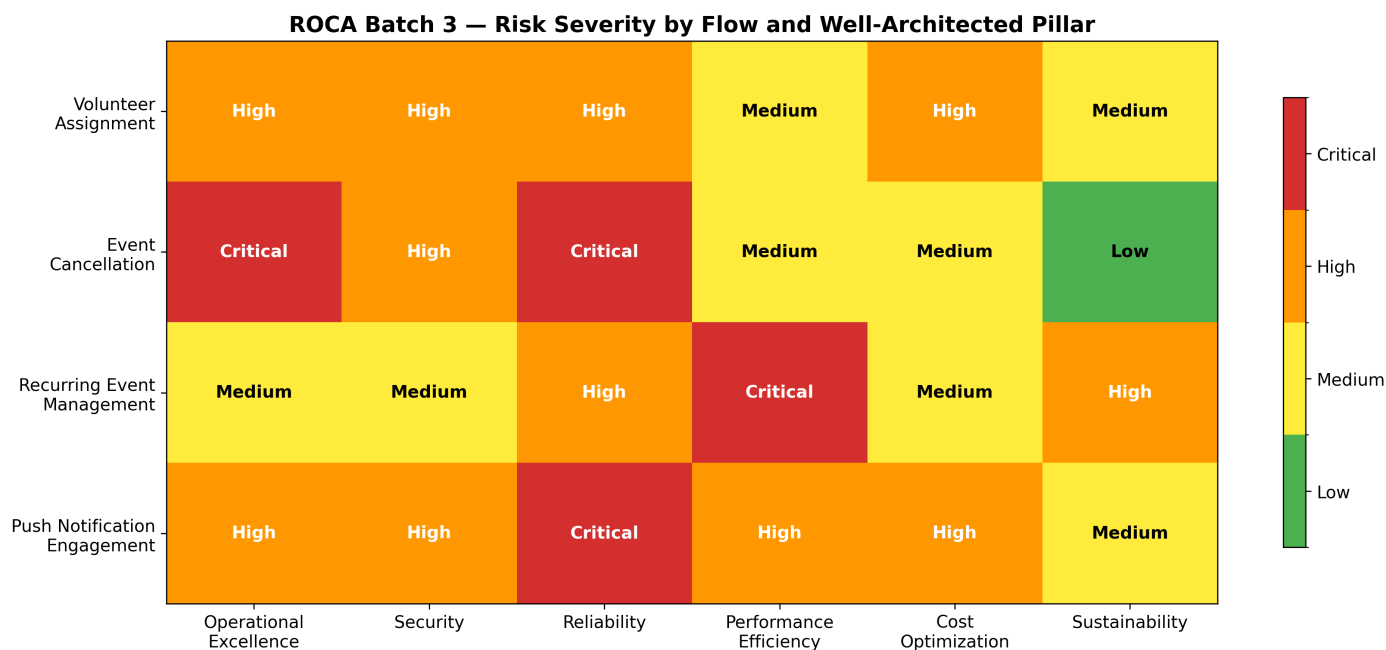


Figure 1: Risk Severity Heatmap

Reliability is the most broadly impacted pillar. The absence of DLQs across all SQS queues means failed messages are silently lost [3]. The Event Cancellation flow lacks saga compensation for partial failures [2]. The Push Notification flow has no systematic bounce handling, risking SES sender reputation degradation [4, 5]. The Recurring Event flow risks Lambda timeout exhaustion from unbounded RRULE expansion [7, 8].

Performance Efficiency is most acute for the Recurring Event flow, where lazy materialization without bounded query windows can produce excessive compute per request [19], and for the Push Notification flow, where Web-Socket idle timeouts and connection duration limits require careful lifecycle management [12]. The Volunteer Assignment flow's auto-schedule algorithm runs without memory right-sizing or Graviton2 optimization [9, 14].

Cost Optimization opportunities center on migrating short-lived orchestrations to Express Workflows at \$1.00 per million versus \$25 per million for Standard [16], enabling Graviton2 for compute-intensive Lambda functions [14], and implementing caching to reduce redundant scheduling computations [15].

Sustainability concerns arise from compute waste in unoptimized Lambda functions and redundant RRULE expansion that could be eliminated through hybrid materialization and caching strategies.

9 Prioritized Remediation Roadmap

The remediation roadmap spans approximately 80–95 engineering days across three priority tiers, sequenced to address critical reliability and security gaps first. The following chart breaks down estimated effort by individual remediation item and priority tier.

Chart Explanation: This chart shows the estimated engineering effort in days for each remediation item, color-coded by priority tier. P0 Critical items (red, Days 1–30) total approximately 27 days and address the most severe reliability and security gaps. P1 High items (orange, Days 31–60) total approximately 26 days and focus on architectural migration. P2 Medium items (blue, Days 61–90) total approximately 24 days and optimize performance, cost, and observability.

9.1 P0 Critical — Days 1–30 (approximately 27 engineering days)

The highest-priority remediation is deploying DLQs on all SQS queues (approximately 3 days), configuring maxReceiveCount redrive policies so failed messages are isolated for investigation rather than lost [3]. WAF and GuardDuty deployment (approximately 5 days) addresses the Batch 1 P0-Critical security gap escalated in Batch 2 for PCI DSS compliance. JWT rotation via Cognito (approximately 4 days) closes the authentication vulnerability carried forward from Batch 1. The Step Functions saga implementation for the Event Cancellation cascade (approximately 10 days) is the largest P0 item, requiring definition of Task states for each cancellation step with corresponding compensating transactions wired via Catch blocks [1, 2], plus integration with Step Functions Wait states for the timed notification cascade. SES bounce handling configuration with separate SNS topics for bounces and complaints,

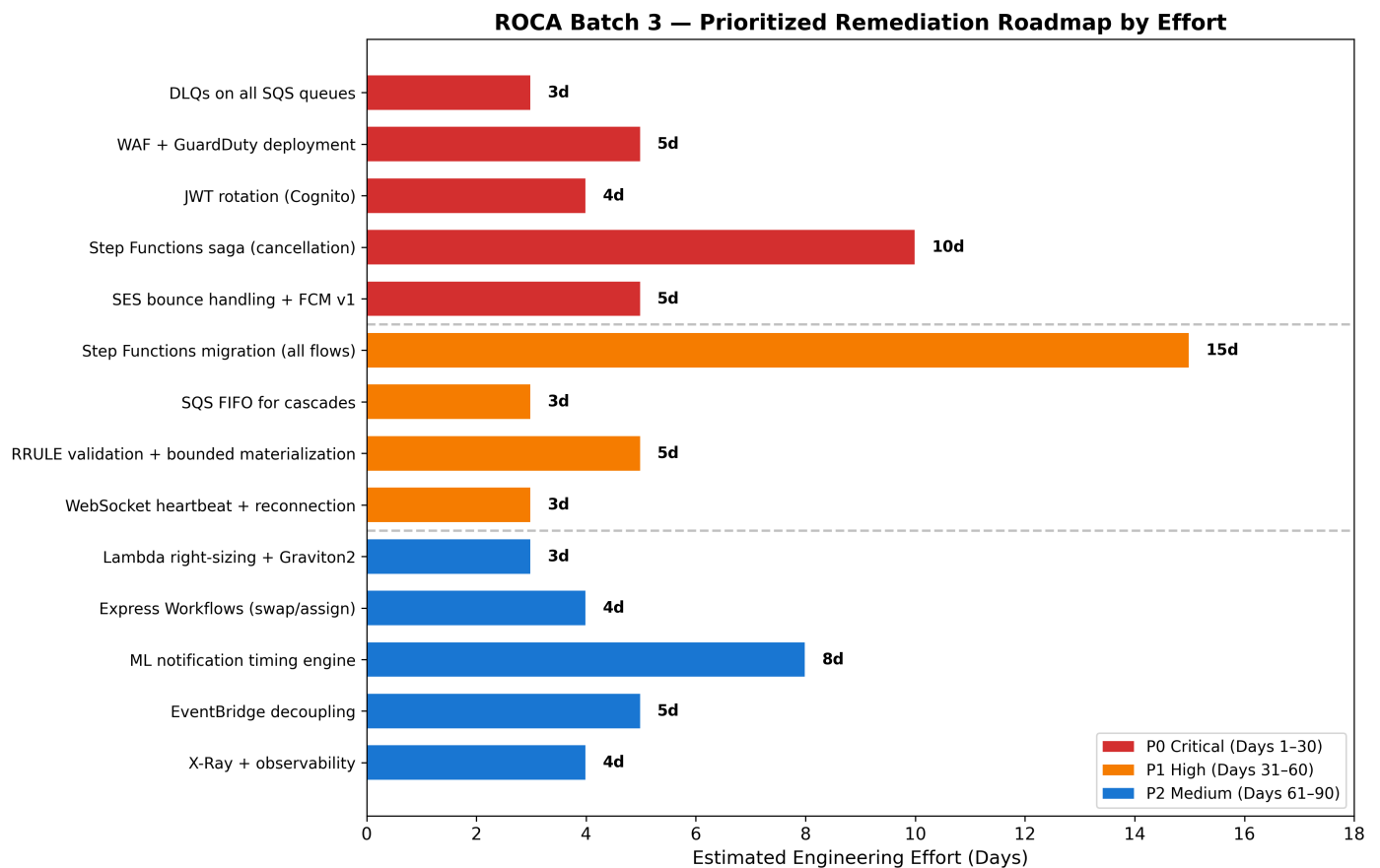


Figure 2: Remediation Roadmap

plus FCM HTTP v1 API migration (approximately 5 days), addresses the Push Notification flow’s critical delivery reliability gaps [4, 6].

9.2 P1 High — Days 31–60 (approximately 26 engineering days)

Step Functions migration for all four flow state machines (approximately 15 days) eliminates the embedded state machine anti-pattern across the Volunteer Assignment, Recurring Event, and Push Notification flows. SQS FIFO queue deployment for ordered cascading operations (approximately 3 days) ensures exactly-once processing for sequential cancellation steps [11]. RRULE input validation and bounded materialization (approximately 5 days) enforces COUNT/UNTIL requirements and caps query-time expansion to 90-day windows [7, 19]. WebSocket heartbeat and reconnection logic (approximately 3 days) implements client-side ping intervals under 10 minutes and exponential backoff reconnection after the 2-hour hard cutoff [12].

9.3 P2 Medium — Days 61–90 (approximately 24 engineering days)

Lambda right-sizing with Graviton2 migration (approximately 3 days) optimizes the auto-schedule algorithm’s compute cost using AWS Compute Optimizer recommendations and arm64 architecture [9, 14]. Express Workflows for swap and assignment orchestration (approximately 4 days) reduces per-invocation cost for sub-5-minute workflows [16]. The ML notification timing engine (approximately 8 days) implements send-time optimization with demographic cohort priors for cold-start users [33]. EventBridge decoupling (approximately 5 days) and X-Ray distributed tracing with observability dashboards (approximately 4 days) address the remaining Batch 1 operational excellence gaps.

Priority	Timeline	Items	Est. Days	Key Dependencies
P0 Critical	Days 1–30	DLQs, WAF/ GuardDuty, JWT rotation, Saga pattern, SES bounce + FCM v1	~27	None — can begin immediately
P1 High	Days 31–60	Step Functions migration (all flows), SQS FIFO, RRULE validation, WebSocket heartbeat	~26	P0 DLQs and saga pattern complete
P2 Medium	Days 61–90	Lambda op- timization, Express Workflows, ML timing, EventBridge, X-Ray	~24	P1 Step Functions migration complete

Table: Remediation Roadmap Summary by Priority Tier

The total estimated effort of 77–95 engineering days (accounting for integration testing and rollback contingency) represents a significant but necessary investment. The P0 tier alone addresses the most severe compounding risks — silent message loss, uncompensated cascade failures, and security exposure — that affect all four middle-tier flows simultaneously. Completing P0 items within the first 30 days will materially reduce ROCA’s operational and compliance risk posture before proceeding to the architectural improvements in P1 and the optimization work in P2.

10 References

1. <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-orchestration.html>
2. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/implement-the-serverless-saga-pattern-by-using-aws-step-functions.html>
3. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html>
4. <https://aws.amazon.com/de/blogs/messaging-and-targeting/handling-bounces-and-complaints/>
5. <https://aws.amazon.com/premiumsupport/knowledge-center/ses-high-bounce-rate/>
6. <https://docs.aws.amazon.com/sns/latest/dg/sns-fcm-authentication-methods.html>
7. <https://github.com/schedulespark/rrule>
8. <https://docs.aws.amazon.com/lambda/latest/dg/configuration-timeout.html>
9. https://docs.aws.amazon.com/en_en/lambda/latest/dg/configuration-memory.html
10. <https://docs.aws.amazon.com/ses/latest/dg/send-email-concepts-deliverability.html>
11. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-fifo-queues.html>
12. <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-execution-service-websocket-limits-table.html>
13. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/cost-and-performance-optimization.html>

14. <https://docs.aws.amazon.com/lambda/latest/dg/foundation-arch.html>
15. [https:// docs.aws.amazon.com/ prescriptive-guidance/ latest/ optimize-costs-microsoft-workloads/ net-serverless.html](https://docs.aws.amazon.com/prescriptive-guidance/latest/optimize-costs-microsoft-workloads/net-serverless.html)
16. <https://aws.amazon.com/blogs/compute/building-cost-effective-aws-step-functions-workflows/>
17. <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html>
18. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/failure-management.html>
19. <https://cli.nylas.com/guides/recurring-calendar-events-api>
20. <https://icalendar.org/iCalendar-RFC-5545/3-8-5-3-recurrence-rule.html>
21. <https://github.com/dateutil/dateutil/issues/641>
22. <https://github.com/jkbrzt/rrule/issues/550>
23. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/data-modeling-schema-recurring-payments.html>
24. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>
25. <https://thoughtbot.com/blog/recurring-events-and-postgresql>
26. <https://www.postgresql.org/docs/current/rules-materializedviews.html>
27. <https://knock.app/manuals/push-notifications/technical-implementation-of-push-notifications>
28. <https://codemia.io/system-design/design-a-push-notification-service/editorial>
29. <https://aws.amazon.com/premiumsupport/knowledge-center/ses-understand-soft-bounces/>
30. https://docs.aws.amazon.com/en_en/sns/latest/dg/mobile-push-notifications-best-practices.html
31. <https://docs.aws.amazon.com/sns/latest/dg/sns-fcm-endpoint-management.html>
32. https://docs.aws.amazon.com/sdk-for-python/v1/reference/clients/sns/operations/get_endpoint_attributes/
33. <https://algomaster.io/learn/ml-system-design/design-notification-relevance-system>
34. <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-known-issues.html>
35. [https:// docs.aws.amazon.com/ apigateway/ latest/ developerguide/ apigateway-how-to-call-websocket-api-connections.html](https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-how-to-call-websocket-api-connections.html)
36. <https://costgoat.com/pricing/amazon-api-gateway>
37. https://aws.amazon.com/api-gateway/pricing/?__=